

P1766R1

Mitigating minor modules maladies

Published Proposal, 2019-07-18

This version:

<http://wg21.link/p1766r1>

Author:

[Richard Smith](#) (Google)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper identifies three minor issues relevant to modules and suggests resolutions

Table of Contents

1 Revision history

1.1 Since R0

2 Typedef names for linkage purposes

2.1 Compiler misbehavior

2.2 Modules impact

2.3 Proposed resolution

2.4 Rationale for fixing for C++20

3 Static assertions and empty declarations in `export`

3.1 Problem

3.2 Proposed resolution

3.3 Ship vehicle

4 Default argument inconsistency

4.1 Problem

4.2 Proposed resolution

4.3 Rationale for fixing for C++20

5 Wording

5.1 Feature test macro

§ 1. Revision history

§ 1.1. Since R0

SG2 voted to apply the first and third sections for C++20; wording for those is provided below. The section section will be the subject of a different paper.

§ 2. Typedef names for linkage purposes

§ 2.1. Compiler misbehavior

```
template<typename T> int X;  
typedef struct {  
    int *n = &X<decltype(this)>;  
} Y;  
Y y;
```

GCC miscompiles this (incorrectly giving `X<Y>` internal linkage).

Clang rejects, giving a hint as to what's going on:

```

<source>:4:3: error: unsupported: typedef changes linkage of anonymous type
} Y;
  ^
<source>:2:15: note: use a tag name here to establish linkage prior to defini
typedef struct {
               ^
               Y

```

This is not an isolated example: lots of constructs within an anonymous struct can trigger a linkage computation, but the linkage computation cannot be performed until *after* we reach the end of the struct and find out whether it has a typedef name for linkage purposes.

The `typedef` keyword doesn't help us here. This case does not have a typedef name for linkage purposes:

```
typedef struct { /* ... */ } *p;
```

... and this case does:

```
struct { /* ... */ } typedef x; // ???
```

§ 2.2. Modules impact

When such a construct appears in a header unit, the problem is exacerbated. It is possible for the compiler to be aware of another definition of a struct from a different translation unit, but for that other definition to not be reachable:

```

// foo.h
#ifndef F00_H
#define F00_H
typedef struct { /*...*/ } X;
#endif

```

```
module;  
#include "foo.h"  
export module A;  
X x;
```

```
export module B;  
import A;
```

```
import B;  
#include "foo.h"
```

In the final translation unit above, the definition of `X` is not (necessarily) reachable, so including a definition of `X` is acceptable. However, a compiler may already know of the definition of `X` in module `A`. This results in a need to "merge" the new definition with the old one. There are various techniques for this, but they typically rely on knowing that the definition is in fact a redeclaration of the already-known definition before anything "too complicated" happens.

§ 2.3. Proposed resolution

Typedef names for linkage purposes exist for C compatibility. Therefore it would be reasonable to restrict their use to C-compatible structures.

We propose disallowing any case where a class is given a typedef name for linkage and its members include anything other than:

- non-static data members with no default member initializers (and no *lambda-expressions* in the type of the member)
- `static_assert` declarations
- empty declarations

This is intended to ensure that structs with typedef names for linkage purposes are always simple enough that no linkage calculation is necessary before the typedef name for linkage is encountered, and likewise that the post-facto "merge" step is simple.

§ 2.4. Rationale for fixing for C++20

This is a bugfix; the issue is longstanding but is expected to become more significant in the presence of modules in C++20.

§ 3. Static assertions and empty declarations in `export`

§ 3.1. Problem

In the Modules TS and all subsequent proposals, there is a restriction that an `export` declaration or `export` block can contain only declarations that introduce at least one name.

For individual `export` declarations, this makes sense:

```
export static_assert(true); // error: what do you mean, 'export'?
```

But for `export` blocks, this rule introduces some awkwardness:

```
export {
    struct Foo { /*...*/ };
    static_assert(std::is_trivially_copyable_v<Foo>); // error: can't export

    struct Bar { /*...*/ };

    template<typename T> struct X { T t; };
    template<typename T> X(T) -> X<T>; // error: can't export this either!

    // ...

#define STR(x) constexpr char x[] = #x;
    STR(foo); // error: can't export empty-declaration ';'
    STR(bar); // error: can't export empty-declaration ';'
#undef X
}
```

To avoid this, the code must be reorganized: the `export` block could be split into pieces, or the `static_assert` and deduction guide could be moved elsewhere (and the redundant `;`s after the macros could be removed).

§ 3.2. Proposed resolution

Remove the rule that a declaration in an `export` block introduces at least one name. Retain the rule for the single-declaration form of `export`.

§ 3.3. Ship vehicle

Following the guidance of SG2, this direction will be pursued for C++23.

§ 4. Default argument inconsistency

§ 4.1. Problem

For non-inline function declarations, C++ permits default arguments to be different in different translation units. Likewise, for templates, C++ permits default template arguments to be different in different translation units:

```
// a.h
int f(int a = 123);
```

```
// b.h
int f(int a = 45);
```

It would be an error if both declarations were included into the same translation unit. But modules creates situations where both declarations may be simultaneously reachable (even though only one of them is visible) and it is unduly burdensome on an implementation to require that only the visible default argument is used, or even that the mismatch is detected and diagnosed.

```
import A;      // indirectly uses a.h, does not export it
import "b.h";
int n = f();   // ??
```

§ 4.2. Proposed resolution

Disallow giving the same function parameter different default arguments in two declarations in the same namespace scope in different translation units, and likewise for default template arguments. No

diagnostic is required unless one such function declaration is reachable where the other is encountered.

Note that we already have a similar rule for inline functions.

§ 4.3. Rationale for fixing for C++20

This fixes an area of underspecification and friction in a new C++20 feature.

§ 5. Wording

Change in 6.2 [basic.def.odr] paragraph 1:

A variable, function, class type, enumeration type, ~~or~~ template, default argument for a parameter (for a function in a given scope), or default template argument shall not be defined where a prior definition is necessarily reachable (10.6); no diagnostic is required if the prior declaration is in another translation unit.

Change in 6.2 [basic.def.odr] paragraph 12:

There can be more than one definition of a class type (Clause 11), enumeration type (9.6), inline function with external linkage (9.1.6), inline variable with external linkage (9.1.6), class template (Clause 13), non-static function template (13.6.6), concept (13.6.8), static data member of a class template (13.6.1.3), member function of a class template (13.6.1.1), ~~or~~ template specialization for which some template parameters are not specified (13.8, 13.6.5), default argument for a parameter (for a function in a given scope), or default template argument in a program provided that no prior definition is necessarily reachable (10.6) at the point where a definition appears, and provided the definitions satisfy the following requirements. There shall not be more than one definition of an entity that is attached to a named module (10.1); no diagnostic is required unless a prior definition is reachable at a point where a later definition appears.

Change in 9.1.3 [dcl.typedef] paragraph 9:

If the typedef declaration defines an unnamed class (or enum), the first *typedef-name* declared by the declaration to be that class type (or enum type) is used to denote the class type (or enum type) for linkage purposes only (6.5). [Note: A typedef declaration involving a *lambda-expression* does not itself define the associated closure type, and so the closure type is not given a name for linkage purposes. -end note] [Example:

```
typedef struct { } *ps, S;  
typedef decltype([]{}) C;
```

-end example] An unnamed class with a typedef name for linkage purposes shall not

- declare any members other than non-static data members, member enumerations, or member classes,
- have any base classes or default member initializers, or
- contain a *lambda-expression*,

and all member classes shall also satisfy these requirements (recursively). [Example:

```
typedef struct {  
    int f() {}  
} X; // error: struct with typedef name for linkage has member
```

-end example]

Change in 9.2.3.6 [dcl.fct.default] paragraph 4:

For non-template functions, default arguments can be added in later declarations of a function in the same scope. Declarations in different scopes have completely distinct sets of default arguments. That is, declarations in inner scopes do not acquire default arguments from declarations in outer scopes, and vice versa. In a given function declaration, each parameter subsequent to a parameter with a default argument shall have a default argument supplied in this or a previous declaration, unless the parameter was expanded from a parameter pack, or shall be a function parameter pack. [Note: A default argument ~~shall not~~ cannot be redefined by a later declaration (not even to the same value) ([basic.def.odr]). -end note] [Example: ...] For a given inline function defined in different translation units, the accumulated sets of default arguments at the end of the translation units shall be the same; ~~see 6.2~~ no diagnostic is required. If a friend declaration specifies a default argument expression, that declaration shall be a definition and shall be the only declaration of the function or function template in the translation unit.

Add two entries to C.5:

Affected subclause: [dcl.typedef]

Change: Unnamed classes with a typedef name for linkage purposes can only contain C-compatible constructs.

Rationale: Necessary for implementability.

Effect on original feature: Valid C++ 2017 code may be ill-formed in this International Standard.

```
typedef struct {  
    void f() {}    // ill-formed; previously well-formed  
} S;
```

Affected subclause: [dcl.fct.default]

Change: A function cannot have different default arguments in different translation units.

Rationale: Required for modules support.

Effect on original feature: Valid C++ 2017 code may be ill-formed in this International Standard, with no diagnostic required.

```
// Translation unit 1  
int f(int a = 42);  
int g() { return f(); }  
  
// Translation unit 2  
int f(int a = 76) { return a; } // ill-formed (no diagnostic required); p  
int g();  
int main() { return g(); }      // used to return 42
```

§ 5.1. Feature test macro

No feature test macro is proposed. Code wishing to be compatible across language standards should avoid the removed functionality.